

005 Week 4 — English Worksheet (Intermediate)

Topic: Petting Zoo — Using Helper Functions · **Course:** Theme Park Creator · **Time:** about 45 minutes

Your park has stalls and amenities. Now it needs a **petting zoo** — a fence around it, six cages inside, and a feeding area.

The outer fence is a fence box. Every cage is a fence box too. Same job, different size. So **one** `build_fence` helper builds all seven.

These are the blocks you use this week:

```
blocks.fill(OAK_FENCE, corner1, corner2, FillOperation.OUTLINE)
blocks.place(OAK_FENCE_GATE, pos(9, 0, 0))
mobs.spawn(CHICKEN, pos(3, 1, 3))
```

`FillOperation.OUTLINE` = the **edge** of the box only — the ground inside is left alone.

`FillOperation.REPLACE` = the **whole solid** box. `blocks.place(block, pos(x, y, z))` puts **one** block in **one** spot. `mobs.spawn(animal, pos(x, y, z))` puts **one** animal in **one** spot.

1 · How to Use a Helper Function

A **helper function** does one job. Other functions **call** it instead of writing that job again.

Four steps, every time:

Step 1 — Find the job that repeats. The zoo fence is a fence box. Each cage is a fence box. That job runs seven times.

Step 2 — Write the job once, in its own function, with a name that says the job: `build_fence`.

Step 3 — Put the changing parts in the brackets. The material changes. The spot and the size change. Those become **parameters**.

Step 4 — Call it from wherever you need that job, with different arguments each time.

```

build_zoo()
├─▶ build_fence(...)           ← the outer fence
├─▶ build_cage(CHICKEN, 2, 2)
│   └─▶ build_fence(...)      ← the SAME helper, smaller
│       └─▶ spawn CHICKEN
└─▶ build_feeding_area(14, 4)

```

Here is the helper. It builds one fence box, anywhere, any size:

```

def build_fence(fence, x, z, size):
    corner1 = pos(x, 0, z)
    corner2 = pos(x + size, 0, z + size)
    blocks.fill(fence, corner1, corner2, FillOperation.OUTLINE)

```

The zoo fence and the cage fences are all built by this one helper. What has to be different each time it is called?

2 · Predict 🧠

Read each function. Write what you think happens.

```
def build_cage(animal, x, z):  
    build_fence(OAK_FENCE, x, z, 2)  
    mobs.spawn(animal, pos(x + 1, 1, z + 1))  
  
build_cage(SHEEP, 6, 2)
```

Which values reach `build_fence` ? Where does the sheep appear, and why is it `x + 1` and not `x` ?


```
build_cage(PIG, 10, 2)  
build_cage(RABBIT, 10, 8)
```

Both calls send the same `x` . What is different about the two cages?

3 · Spot the Bug 🐛

Each block is broken. Read what it should do, rewrite it so it works, then explain why the original was wrong.

Bug A — This should build a cage with a sheep inside. The sheep appears, but it stands in an **open field** with no fence around it.

```
def build_cage(animal, x, z):  
    mobs.spawn(animal, pos(x + 1, 1, z + 1))  
  
build_cage(SHEEP, 6, 2)
```

Hint: the helper exists, but nobody calls it.

Write the fixed code:

Why was it wrong? Why does your fix work?

Bug B — Six cages should stand in six different spots. Instead all six animals appear in **one** cage, on top of each other.

```
build_cage(CHICKEN, 2, 2)  
build_cage(COW, 2, 2)  
build_cage(PIG, 2, 2)  
build_cage(SHEEP, 2, 2)  
build_cage(RABBIT, 2, 2)  
build_cage(LLAMA, 2, 2)
```

Hint: the helper works. The arguments do not.

Write the fixed code:

Why was it wrong? Why does your fix work?

Bug C — The outer fence should be a ring around the zoo, leaving everything inside it standing. Instead the chicken's cage **disappears** the moment the outer fence goes up, and the chicken is left standing in the open.

```
def build_fence(fence, x, z, size):  
    corner1 = pos(x, 0, z)  
    corner2 = pos(x + size, 0, z + size)  
    blocks.fill(fence, corner1, corner2, FillOperation.HOLLOW)  
  
def build_zoo():  
    build_cage(CHICKEN, 2, 2)  
    build_fence(OAK_FENCE, 0, 0, 18)
```

Hint: **HOLLOW** clears the middle of the box to air. **OUTLINE** leaves the middle alone. One word, in one place, fixes all seven fences.

Write the fixed code:

Why was it wrong? Why does your fix work?

4 · Build the Petting Zoo 🐑

In your homework world, write **one** helper and three functions that call it:

```
def build_fence(fence, x, z, size):
def build_cage(animal, x, z):
def build_feeding_area(x, z):
def build_zoo():
```

`build_fence` builds one fence box with `FillOperation.OUTLINE` .

`build_cage` calls `build_fence` , then spawns its animal inside.

`build_feeding_area` fills `HAY_BLOCK` for food and places `WATER` for drink.

`build_zoo` calls `build_fence` once for the outer fence, places an `OAK_FENCE_GATE` so visitors can walk in, calls `build_cage` **six** times with **six different animals**, and calls `build_feeding_area` once.

Only `build_fence` may contain a fence `blocks.fill` . The cages and the outer fence all come from the helper.

Animals: `CHICKEN` · `COW` · `PIG` · `SHEEP` · `RABBIT` · `LLAMA`

Cage spots that fit inside an 18 × 18 zoo — front row and back row:

```
(2, 2)    (6, 2)    (10, 2)    feeding area: (14, 4)
(2, 8)    (6, 8)    (10, 8)    gate: pos(9, 0, 0)
```

Write your helper, your three functions, and the call to `build_zoo` :

5 · Show Your Work 📷 🎥

Record **one video** — one take, no stopping (a phone is fine). Show these in order:

1 · Your code. Scroll through it. Say what each part does.

2 · Your build. Point the camera. Name the parts.

Fill the blanks:

Today I built _____.

I built it using this code: _____.

In this code I used _____.

The hardest part was _____.

That part was hard because _____.

The most fun part was _____.

Something new I learned was _____.

Write your lines here, then say them in your video.

Submit ✓

Send this worksheet + your video to teacher on KakaoTalk.